Project Information

Group ID	16	**Semester (SS2?)**
1. Student Name	Ayham Alhasan	SS26
2. Student Name	Samet Yilmaz	SS26
3. Student Name	Robert-Bogdan Fesko	SS26
4. Student Name		

Project Title	Chat Application
Project Description	A distributed chat application implemented in Java 21. Multiple clients join a shared chat room and exchange messages through a group of replicated servers. The system is scalable in the number of clients and tolerates the crash of any single server, including the leader. Leader election uses the Bully algorithm (highest-ID-wins). The system uses UDP broadcast for dynamic discovery and UDP heartbeats for failure detection, and runs on three physical hosts on a local network with no external middleware.
	GitHub/GitLab/BitBucket etc. URL:
Code Repository Link	https://github.com/robertfeo/chatapp-ds

Architectural Model

Architecture Type

- (●) Client-Server
- ( ) Peer-to-Peer
- ( ) Hybrid

Mechanism

Communication

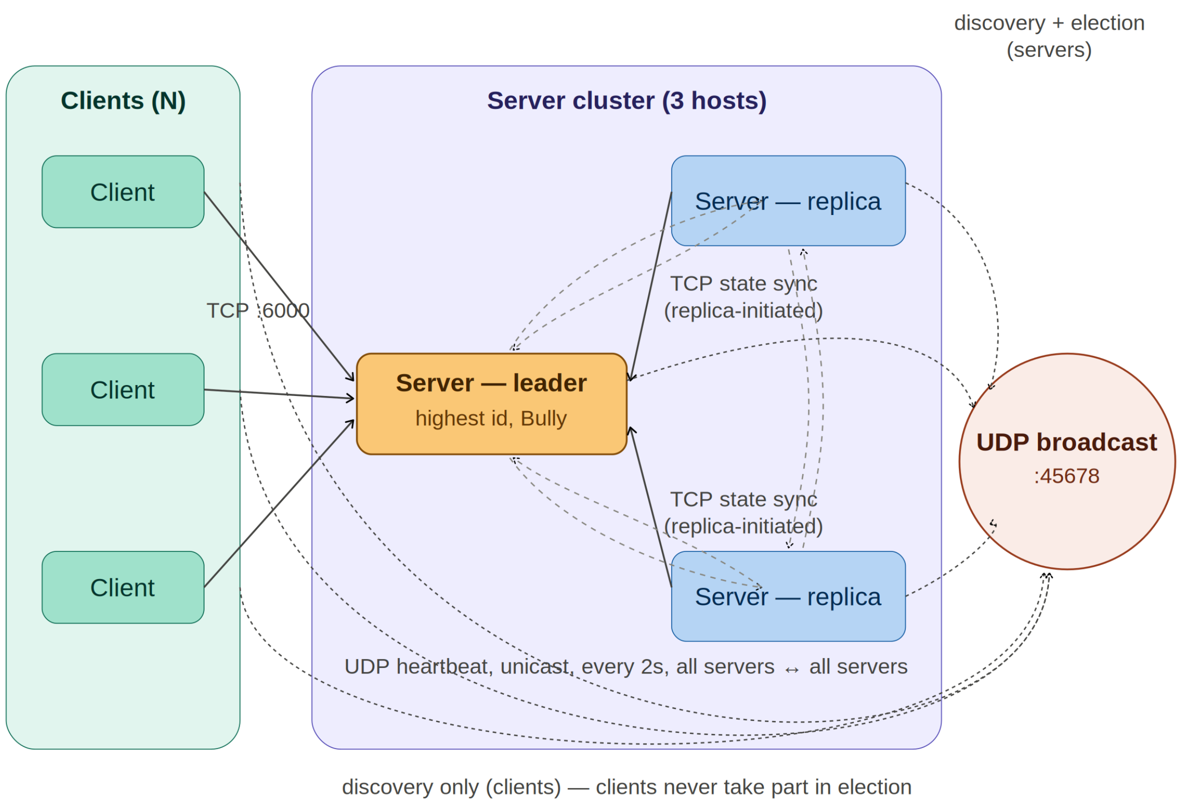
	Explain briefly what it is used for:
TCP	TCP carries all chat traffic and replication. Clients connect to the current leader exclusively over TCP (port 6000) to send and receive chat messages; the leader fans accepted messages out to every connected client over the same connections. The leader also replicates state to the replica servers over TCP, pushing chat-history deltas (StateSync) after each accepted message; a (re)joining replica requests a full history snapshot (HistoryRequest/HistorySnapshot) over the same connection. One TCP port serves both client and replica sessions; the role is determined from the first message received on the connection.
	Explain briefly what it is used for:
UDP	UDP is used for everything that is not chat traffic. Dynamic discovery (DISCOVERY_HELLO/DISCOVERY_REPLY) and the Bully leader-election messages (ElectionInquiry, Answer, IAmLeader) are sent by UDP broadcast on the discovery port (45678), since these messages must reach all peers on the subnet. Heartbeats are sent by UDP unicast directly to each known peer every 2 seconds to monitor liveness.
	Explain briefly what it is used for:
Other	None.

Concurrency

	Explain briefly what it is used for:
Multithreading	Java 21 virtual threads (Thread.ofVirtual()) are used instead of classic platform threads. Each server runs a dedicated virtual thread for the TCP accept loop and one virtual thread per accepted client/replica connection (so every chat client and every replica connection is served concurrently without blocking the others). The UDP discovery listener also runs on its own virtual thread. Scheduled, recurring tasks (heartbeats, election timeouts, discovery re-announce) run on small dedicated single-thread ScheduledExecutorServices. Shared mutable state (chat history, group view, connection sets) is protected with
	Explain briefly what it is used for:
Multiprocessing	Not used. Concurrency within a single server process is handled entirely via threads (see Multithreading); separate OS processes are only used in the sense that each of the three servers and each client is its own JVM process on its own host, but this is the distributed deployment, not an intra-node multiprocessing mechanism.

Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.

System Architecture Desin
Include a clear diagram of your system architecture (key components and connections between them).



Dynamic Discovery of Hosts

Discovery Mechanism

- [ ] Client discovers server
- [x] Server discovers servers
- [ ] Other. Explain briefly:

There is no central registry; every server builds its own local group view from received broadcasts. On startup, a server broadcasts DISCOVERY_HELLO with its (id, host, port). Existing servers add it to their local group view and reply by unicast with DISCOVERY_REPLY carrying their own (id, host, port) and the current leader id. A new client follows the same pattern (broadcasts a hello, learns the leader id from replies, then connects directly to the leader via TCP), but clients are never added to a server's group view, since they have no listening port and never send heartbeats.

Discovery Implemented

- [x] Broadcast
- [ ] Multicast
- [ ] Other. Explain briefly:

UDP broadcast on the local subnet, port 45678 (DISCOVERY_HELLO and DISCOVERY_REPLY). Sent to every live network interface's broadcast address (plus a configured fallback), so the discovery message still reaches the LAN even on hosts with several virtual/VPN adapters.

Discovery Occurs

- [x] When system starts
- [ ] Whenever new component comes in
- [x] Other. Explain briefly:

Every server (and client) announces itself once on startup, and then re-announces itself every 10 seconds for as long as it runs, so that late joiners are eventually discovered by everyone even if they missed the initial broadcast.

Voting

Voting Implemented Using

- (●) Bully Algorithm
- ( ) LeLann-Chang-Roberts Algorithm
- ( ) Hirschberg-Sinclainr Algorithm

Group View Used

- ( ) No
- (●) Yes. Explain briefly what the group view is used for:

Each server keeps a local group view of known peer servers (id, host, port). It is used (1) by the election logic to determine which ids are alive and therefore eligible to win (highest-id-wins), (2) by the heartbeat service to know which peers to send heartbeats to, and (3) by the replica connector to look up the current leader's address when (re) connecting after a leader change.

Nodes Identified Using

- ( ) IP addresses / IP addresses + ports
- ( ) UUIDs
- (●) Other (describe ID format and reasons for using it):

Each server generates a random numeric id (32-bit int) at startup. This is used rather than a UUID because the Bully algorithm needs ids that are totally ordered and cheaply comparable (highest-id-wins); a random numeric id from a large space makes collisions negligible while keeping the election comparison a simple integer max. IP/port pairs are tracked separately per peer for addressing, but are not used as the election id.

Election Starts

- [x] When system starts
- [ ] When a new server joins
- [x] When the leader fails

Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.

Fault Tolerance	
Faults Tolerated	☑ Crash Faults (Leader Server, Regular Server, Client) ☐ Omission Faults ☐ Byzantine Faults
Fault Detection	Explain who sends heartbeats to whom, their frequency, and retries: Every server sends a UDP heartbeat by unicast to every other known server (from its local group view) every 2 seconds. Each server tracks the last-seen timestamp per peer; if no heartbeat has been received from a peer for more than 6 seconds (i.e. roughly 3 missed heartbeats), that peer is declared dead, removed from the local group view, and a callback fires. If the dead peer was the current leader, this callback immediately starts a new Bully election on the surviving server. There is no explicit retry/acknowledgement for individual heartbeats (they are fire-and-forget UDP); robustness instead comes from the periodic resend every 2 seconds and the 6-second grace window before a peer is considered dead.
Recovery Strategy	Server (incl. leader) crash: surviving servers detect the missing heartbeat after 6s and remove the peer from their group view. If the dead peer was the leader, every surviving server starts a Bully election; the highest remaining id wins and broadcasts IAmLeader. All clients and replicas are notified of the new leader, replicas reconnect their outgoing TCP link to the new leader and request a full history snapshot, and clients rediscover the new leader via UDP broadcast and reconnect automatically. Because the leader continuously pushes chat-history deltas to replicas after each accepted message, the newly elected leader already holds the full chat history, so no data is lost on failover. Client crash: the client's TCP connection drops and its session is removed from the leader's active client list; no special recovery is needed since the client holds no server-side state. Client-side reconnect: if a client's connection is lost (e.g. because the leader it was talking to crashed), it buffers locally typed messages, re-runs discovery to find the new leader, reconnects automatically, and then sends the buffered messages.

Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.

Make sure the final file is a non-editable PDF (save or export as a PDF).